

PhotoSorter – Entwicklungsplan (Version 1.0)

Ziel

Das Projekt wird in kleinen, vollständig lauffähigen Schritten entwickelt.

Nach jeder Phase muss das Programm kompilierbar und testbar sein.

Es dürfen keine unfertigen Features in den Hauptbranch gelangen.

Phase 1

Projekt aufsetzen

- Avalonia-Projekt erstellen
- MVVM einrichten
- Dependency Injection
- Logging
- Konfiguration

Ergebnis:

Leeres Fenster.

Phase 2

Grundlayout

- Toolbar
- Thumbnail-Leiste
- Bildbereich
- Statusleiste

Noch keine Funktionen.

Phase 3

Ordner laden

- Ordner auswählen
- rekursiv Bilder finden
- Bildliste erzeugen

Ergebnis:

Dateien werden erkannt.

Phase 4

Bildanzeige

- erstes Bild anzeigen
- nächstes Bild
- vorheriges Bild

Keine Animationen.

Phase 5

Thumbnail-System

- Vorschaubilder erzeugen
 - Thumbnail-Leiste
 - aktuelle Auswahl markieren
-

Phase 6

Zoom

- Mausrad
 - Pan
 - Bild einpassen
 - 100 %
-

Phase 7

Hotkeys

- links
 - rechts
 - Undo
 - Vollbild
 - konfigurierbar
-

Phase 8

Sortiersystem

- Entscheidungen speichern
- Bilder verschwinden aus der Liste
- Zähler aktualisieren

Noch keine Dateioperationen.

Phase 9

Undo

- beliebig viele Schritte zurück
-

Phase 10

Projektdatei

Speichern

Laden

Automatische Wiederherstellung

Phase 11

Dateioperationen

Nach Abschluss:

Dateien verschieben

Papierkorb unterstützen

Fehlerbehandlung

Phase 12

Animationen

Swipe

Thumbnail-Animation

Sanfte Übergänge

Phase 13

Performance

Asynchrones Laden

Thumbnail Cache

Bild Cache

Speicheroptimierung

Phase 14

RAW-Unterstützung

Preview laden

Große RAW-Dateien testen

Phase 15

Einstellungen

Hotkeys

Dark Mode

Animationen

Ordner merken

Phase 16

Fehlerbehandlung

Beschädigte Bilder

Fehlende Dateien

Zugriffsfehler

Projektwiederherstellung

Phase 17

Feinschliff

Icons

Tooltips

Tastenkürzel

Fenstergrößen

Animationen optimieren

Phase 18

Version 1.0

Code bereinigen

Kommentare

Dokumentation

Tests

Release

Coding-Richtlinien

- MVVM strikt einhalten.
- Keine Businesslogik in Views.
- Alle Services über Interfaces.
- Asynchron arbeiten, wo sinnvoll.
- Kleine, verständliche Methoden.

- Jede neue Funktion durch Unit-Tests absichern, sofern sie keine reine UI-Logik ist.
 - Keine toten oder auskommentierten Codepfade.
-

Definition of Done

Eine Aufgabe gilt erst als abgeschlossen, wenn:

- sie kompiliert,
 - keine Compilerwarnungen erzeugt,
 - getestet wurde,
 - dokumentiert ist,
 - bestehende Funktionen nicht beeinträchtigt,
 - sauber in MVVM integriert wurde.
-

Langfristige Erweiterungen (Version 2.x)

- Mehr als zwei Zielordner
- Frei konfigurierbare Aktionen
- Drag-and-Drop-Sortierung
- EXIF-Anzeige
- GPS-Karte
- Bewertungen (1–5 Sterne)
- Farbmarkierungen
- KI-Vorsortierung (unscharfe Bilder, Duplikate etc.)
- Netzwerkordner/NAS
- Stapelumbenennung
- Export der Sortierergebnisse
- Plugins für zusätzliche Bildformate

Der Quellcode soll von Beginn an so strukturiert sein, dass diese Erweiterungen ohne größere Umstrukturierungen ergänzt werden können.